Handout 2
Verified Union-Find

Software Verification

Nova School of Science and Technology
Mário Pereira `mjp.pereira@fct.unl.pt`

Version of November 8, 2026

The goal of this handout is to verify a Why3 implementation of union-find. Union-find is a very common data structure and algorithm that allow one to efficiently encode set partition problems and equivalence classes. A good description of the behavior of union-find can be found on the Wikipedia page: `https://en.wikipedia.org/wiki/Disjoint-set_data_structure`. An even better presentation of union-find is given by R. Sedgewick and K. Wayne in their masterpiece *Algorithms* [1, Chapter 1.5]. For reference, we provide a complete OCaml implementation of union-find in Appendix A.

During this handout you are supposed to complete the specification and proof given in the companion `.mlw` file. This includes:

- pre-conditions (*i.e.*, `requires` clauses)
- post-conditions (*i.e.*, `ensures` clauses)
- termination measures (*i.e.*, `variant` clauses)
- loop invariants
- type invariants
- filling in pieces of code

The union-find data type and corresponding operations are already implemented. **Do not change such implementations. Only fill in the missing pieces of code, where asked**.

Your submission must contain the following items:

- The proposed skeleton file `union_find.mlw` completed with your solution
- The content of the sub-directory `union_find` generated by Why3. In particular, this directory **must contain** session files `why3session.xml` and `why3shapes.gz`.
- A PDF document named `report.pdf` in which you report on your work.

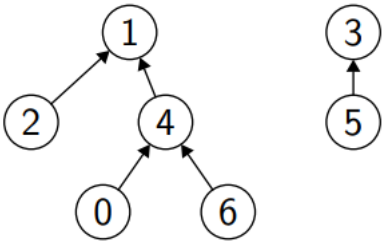
The report can be written either in Portuguese or English. The structure should follow the sections and the questions of this document. For each question, detail your approach, focusing in particular on the design choices that you made regarding the implementations and specifications. In particular, function specification and loop invariants that you added should be explained in your report: what they mean and how they help to complete the proof.

You can use the following structure to answer a question: *"For this function, I propose the following implementation: [give pseudo-code]. The contract of this function is [give a copy-paste of the contract]. It captures the fact that [rephrase the contract in natural language]. To prove this code correct, I need to add extra annotations [give the loop invariants, etc.] capturing that [rephrase the annotations in english]. This invariant is initially true because [explain]. It is preserved at each iteration because [explain]. The post-condition then follows because [explain]."*

The reader of your report should be convinced at each step that the contracts are the right ones, and should be able to understand why your program is correct, *e.g.*, why a loop invariant is initially true, why it is preserved, and why it suffices to prove the post-condition. It is legitimate to copy-paste parts of your Why3 code in the report, yet you should only copy the most relevant parts, not all of your code. The same approach applies to function contracts, in particular for recursive functions: why the contract is preserved for every recursive call and how the body of the function, plus the pre-condition, imply the post-condition.

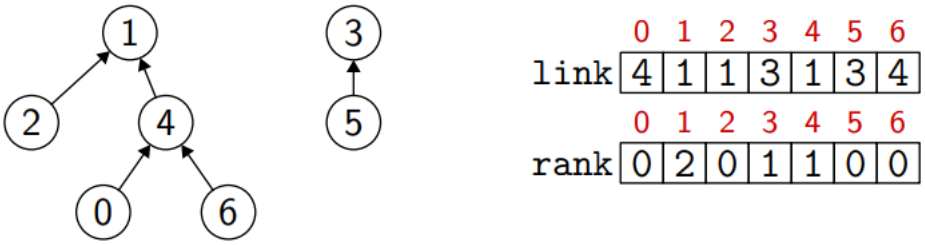
1 Data Type Implementation

Union-find is an efficient implementation, both in space and time, of disjoint classes. The principle behind the union-find data structure is simple: in each class, the elements are chained in such way that all paths lead to a single element, that is the *representative* of that class. For instance, a partition of the set $\{0, 1, 2, 3, 4, 5, 6\}$ into the two classes $\{0, 1, 2, 4, 6\}$ and $\{3, 5\}$ could be as follows:



Here, the representative elements are 1 and 3, respectively. The `find` operation follows the links, until it reaches the representative element. The `union` operation first finds out the representative elements of the two classes, using `find`, and then makes one point to the other.

To make things simpler, but not too simple, we can assume that we are only interested in partitions of the set $\{0, 1, ..., N - 1\}$. In that case, the data structure can be readily implemented with two arrays `link` and `rank`.

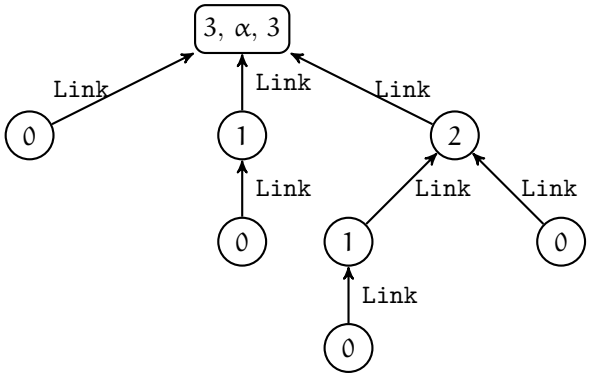


A representative element, such as 1 and 3 here, is linked to itself. In the array `rank`, values are only meaningful for representative elements.

Our Why3 implementation declares the union-find data structure as an Abstract Data Type (ADT) with fields containing the value of N, the two arrays, and a ghost field mapping each element to the representative element of its class. The latter is notably used to specify the operations.

In this handout, we also need to prove the termination of iterative/recursive functions. In order to do so, we add two extra fields to the union-find ADT:

- `dst`, a map that stores, a *distance* value associated to each element of an equivalence class. When following the `link` array, such a distance strictly increases, as we illustrate in the following diagram:



In the above figure, the labels in the nodes represent the values assigned to each one by the `dst` map. The distance values we assign to each element are of no particular interest, as long as they increase until we reach the representative of class.

- `maxd`, a *maximal* value as an upper bound for distances.

Finally, just to make specification a bit easier, we also add a ghost field `size` that stands for the number of elements in the partition.

2 Union-Find Invariants

The union-find data structure invariants can be expressed, on top of the aforementioned fields, as follows:

1. The lengths of the two arrays, `link` and `rank`, are the same an equal to `size`.
2. All elements in the array `link` are less than `size`.
3. For every elements `x` in the partition, `rep x` is less than `size`.
4. The `rep` map encodes an *idempotent* relation. In other words, for every element `x` in the partition we have `rep (rep x) = rep x`.
5. For every element `x` in the partition, `x` is equal to `link[x]` if and only if `x` is equal to `rep x`.
6. For every element `x` in the partition, if `x` is different from `link[x]` then `dst x` is `strictly` less than `dst (link[x])`.
7. For every element `x` in the partition, if `x` is different from `link[x]` then `rep x` is equal to `rep (link[x])`.
8. For every element `x` in the partition, `dst x` is less than or equal to `maxd`.

Exercise 1. Complete the definition and specification of predicate `Valid`, according to the given invariants. □

3 Verified Union Find Operations

3.1 Create operation

The `create` for this ADT is fairly simple: it takes as input an integer `n` and initializes every array and natural field accordingly.

Exercise 2. Complete the specification of `create`. □

3.2 Find operation

Exercise 3. Complete the specification of function `find_it`. Pay particular attention to the termination of `while` loop in this function.

Finally, this function should return the representative element of the argument `x`. The returned value, `r`, should respect the following two properties:

- `dst r` $\geq$ `dst x`
- `r` must be the representative element of `x`, according to map `rep`.

□

Exercise 4.

Based on the implementation of function `find_it`, implement a recursive function `find` with the same purpose of finding the representative of an element in the partition. The specification of both functions should be as close as possible. Pay particular attention to the termination of this function.

However, this recursive function should implement the classic technique of *path compression*. □

3.3 Union operation

Exercise 5. Complete the specification of function `union`. This function should return a *ghost* value `old_rep` which respects the following two properties:

- It is either the former representative of `x` or the former representative of `y`.
- For every element w in the partition, if the representative of w in the pre-state (*i.e.*, before calling the function) is equal to the representative of `x` or `y`, then the representative of w is now `old_rep`. Otherwise, the representative of w does not change.

□

Exercise 6. For each branch of the `union` function, return the correct representative element. □

4 Conclusions

You should end your report with a (small) conclusion that summarizes your achievements and explain the issues you couldn't solve if any.

References

[1] Robert Sedgewick and Kevin Wayne. *Algorithms, 4th Edition.* Addison-Wesley, 2011.

A OCaml Implementation of Union Find

```
type t = {
  link: int array;
  rank: int array;
}

let create (n: int) : t = {
  link = Array.init n (fun (i: int) -> i);
  rank = Array.make n 0
}

let rec find (x: int) (uf: t) : int =
  assert false

let find_it (x: int) (uf: t) : int =
  let y = ref uf.link.(x) in
  while not (!y = uf.link.(!y)) do
    y := uf.link.(!y)
  done;
  !y

let union (i1: int) (j: int) (uf: t) : unit =
  let rep_i = find i1 uf in
  let rep_j = find j uf in
  if not (rep_i = rep_j)
  then begin
    if uf.rank.(rep_i) > uf.rank.(rep_j)
    then uf.link.(rep_j) <- rep_i
    else begin
      uf.link.(rep_i) <- rep_j;
      if uf.rank.(rep_i) = uf.rank.(rep_j)
      then uf.rank.(rep_i) <- uf.rank.(rep_i) + 1
    end end
```